

Table football ball-tracking

USL Final Presentation

Richard B. Rübschläger M. Zain Afzal

University of Siegen

August 30, 2026

Table of Contents

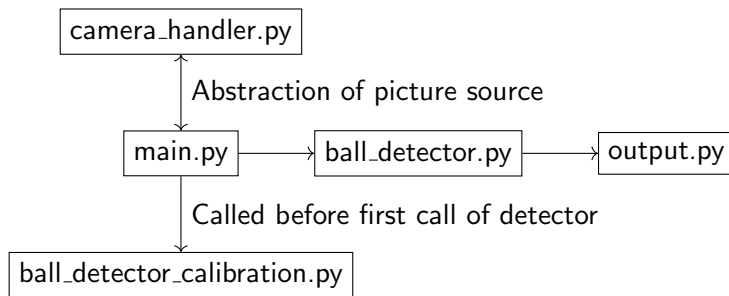
- 1 Goal: Ball Tracking
- 2 Program
 - Program Flow
 - Corner Detection
 - Challenges with corner detection
 - Perspective Transformation
- 3 I²C
 - I²C Communication & Protocol Implementation
 - Challenges & Troubleshooting
- 4 Q&A

Goal: Ball Tracking



Challenge: Track ball position during game and output via I²C

Program Flow



Corner Detection

- Algorithm for top-left corner, but applies to all corners when adjusted for direction
 - Morphological opening used to clean greyscale picture before algorithm
 - Use Manhattan-distance to find the closest pixel



- 1 Find top-left corner of support beam in greyscale picture
 - 2 Find top-left corner of wall in greyscale picture
 - 3 Find top-left corner of field in selection of green colour
- Perform manual correction, in case automatic detection fails
 - Use `cv2.getPerspectiveTransform()` to calculate projection matrix from field corners

Challenges with corner detection

- During game
 - Hands above or in the field.
 - Low FPS
- During calibration
 - Different lighting conditions are very effective at throwing off greyscale thresholding.
 - Bright objects (and people) surrounding the table can be detected as corners.
 - The silver rods throw off brightness based corner detection, prevent morphological closing and opening.
- In general
 - The camera was moved during development. Test pictures taken for development before the camera was moved are no longer accurate.

Perspective Transformation

- ❶ Select orange area from HSV picture using `cv2.inRange()`
- ❷ Check size of area
 - size is zero: No ball on field
 - normal: Ball found
 - very large: Hand over field
 - `M = cv2.moments`
 - `M["m00"]` provides size
- ❸ Calculate center of orange area
 - `int(M["m10"] / M["m00"])` to get x-coordinate
 - `int(M["m01"] / M["m00"])` to get y-coordinate
 - Future Idea: Detect multiple balls
- ❹ Project center coordinates from image space to field space
 - Use previously calculated perspective projection matrix
 - Apply Translation from top-left alignment to center alignment

I²C Communication & Protocol Implementation

- Hardware & Bus Configuration:

Bus & Physical Pins: Raspberry Pi default I²C bus via GPIO 2 (Pin 3) and GPIO 3 (Pin 5).

Target Slave Groups: Broadcasting to address 0x42 (Display) and 0x43 (Feedback).

Baud Rate: 9600 Baud

- Fault-tolerant state handling:

No-Ball Sentinel: Transmits (0x7FFF, 0x7FFF) when ball is lost or occluded.

Goal Cooldown: Suppresses goal re-triggering for 2.0s to prevent duplicate score events.

Data Packet Structure (Little-Endian)

Bytes	Field Name	Data Type	Description
0-3	Current X,Y	2x uint16	mm from field center
4-7	Previous X,Y	2x uint16	
8-11	Field dimensions	2x uint16	1200mm x 680mm
12-15	Ball speed	float32	mm/s
16	Last possession	uint8	0x00=L, 0x01=R
17	Current score	uint8	Upper Nibble: Left

Challenges & Troubleshooting

- Bus Identification & Slave Acknowledgement:

Issue: IO-error caused by hardware NACKs when addressing 0x42 and 0x43.

Resolution: Diagnosed activity with `i2cdetect -y` to isolate active addresses and verified routing.

- Missing ball coordinate handling:

Issue: Coordinates are `None` when ball is missing or occluded.

Resolution: Transmitting Sentinel `0x7FFF` when a coordinate is `None`.

Q&A

Ask your questions